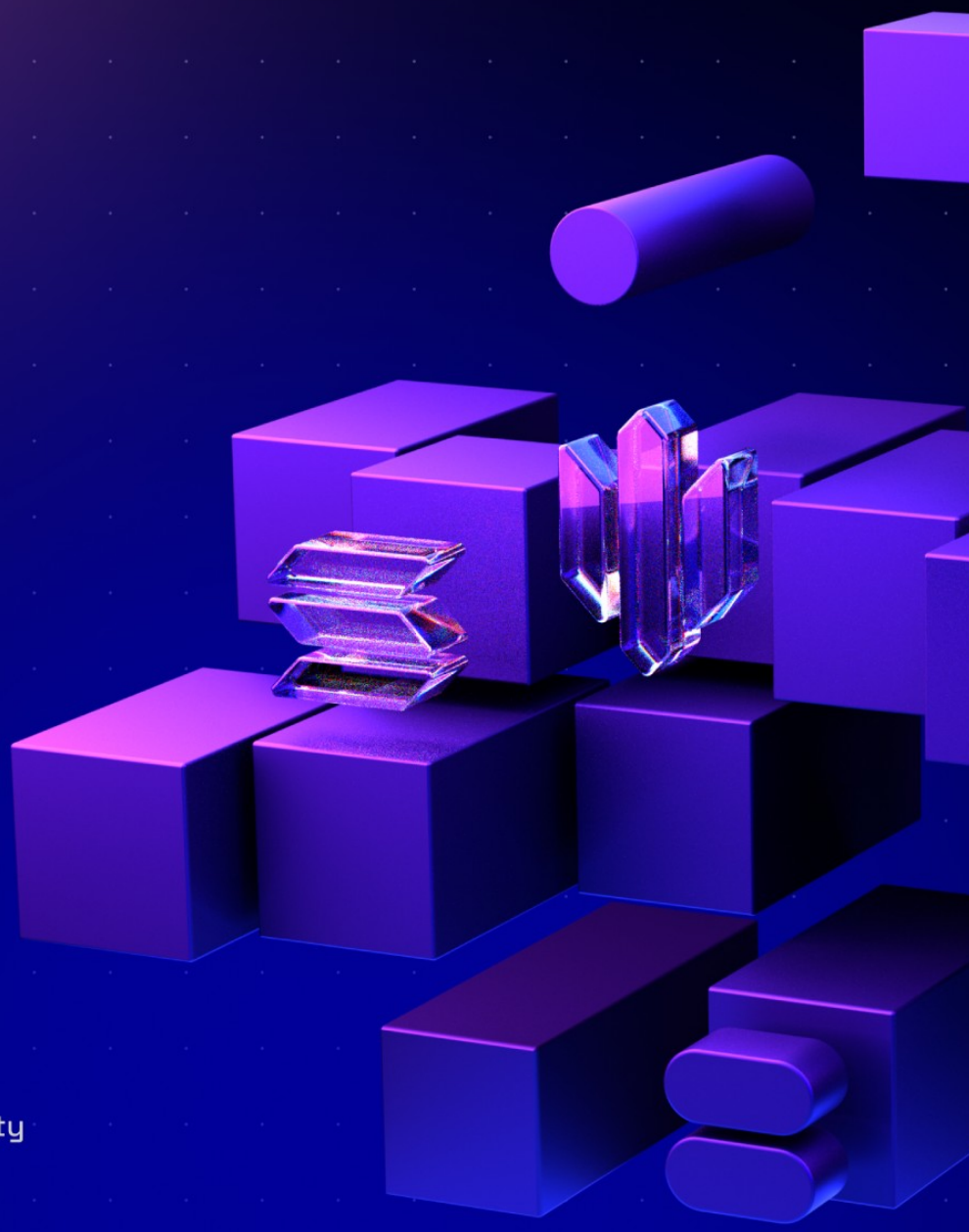
ackee
blockchain security

Jpegs.fun

Launchpad

7.1.2025



Ackee Blockchain Security

Contents

1. Document Revisions . . . 4
2. Overview . . . 5
   2.1. Ackee Blockchain Security . . . 5
   2.2. Audit Methodology . . . 6
   2.3. Finding Classification . . . 7
   2.4. Review Team . . . 9
   2.5. Disclaimer . . . 9
3. Executive Summary . . . 10
   Revision 1.0 . . . 10
   Revision 2.0 . . . 12
   Revision 2.1 . . . 13
4. Findings Summary . . . 14
Report Revision 1.0 . . . 17
   Revision Team . . . 17
   System Overview . . . 17
   Trust Model . . . 17
   Findings . . . 17
Report Revision 2.0 . . . 40
   Revision Team . . . 40
   System Overview . . . 40
   Trust Model . . . 40
   Findings . . . 40
Report Revision 2.1 . . . 55
   Revision Team . . . 55
   System Overview . . . 55
   Trust Model . . . 55

Appendix A: How to cite 56

1. Document Revisions

1.0	Final Report	03.12.2024
2.0	Final Report	20.12.2024
2.1	Fix Review	07.01.2025

2. Overview

This document presents our findings in reviewed contracts.

2.1. Ackee Blockchain Security

Ackee Blockchain Security is an in-house team of security researchers performing security audits focusing on manual code reviews with extensive fuzz testing for Ethereum and Solana. Ackee is trusted by top-tier organizations in web3, securing protocols including Lido, Safe, and Axelar.

We develop open-source security and developer tooling Wake for Ethereum and Trident for Solana, supported by grants from Coinbase and the Solana Foundation. Wake and Trident help auditors in the manual review process to discover hardly recognizable edge-case vulnerabilities.

Our team teaches about blockchain security at the Czech Technical University in Prague, led by our co-founder and CEO, Josef Gattermayer, Ph.D. As the official educational partners of the Solana Foundation, we run the School of Solana and the Solana Auditors Bootcamp.

Ackee's mission is to build a stronger blockchain community by sharing our knowledge.

Ackee Blockchain a.s.
Rohanske nabrezi 717/4
186 00 Prague, Czech Republic
https://ackee.xyz
hello@ackee.xyz

2.2. Audit Methodology

The Ackee Blockchain Security auditing process follows a routine series of steps:

1. **Code review**
    a. High-level review of the specifications, sources, and instructions provided to us to make sure we understand the project's size, scope, and functionality.
    b. Detailed manual code review, which is the process of reading the source code line-by-line to identify potential vulnerabilities. We focus mainly on common classes of Solana program vulnerabilities, such as:

       missing ownership checks, missing signer authorization, signed CPI of unverified programs, cosplay of Solana accounts, missing rent exemption assertion, bump seed canonicalization, incorrect accounts closing, casting truncation, numerical precision errors, arithmetic overflows or underflows.

    c. Comparison of the code and given specifications, ensuring that the program logic correctly implements everything intended.
    d. Review of best practices to improve efficiency, clarity, and maintainability.
2. **Testing and automated analysis**
    a. Run client's tests to ensure that the system works as expected, potentially write missing unit or fuzzy tests using our testing framework Trident.
3. **Local deployment + hacking**
    a. The programs are deployed locally, and we try to attack the system and break it. There is no specific strategy here, and each project's attack attempts are unique to its implementation.

2.3. Finding Classification

A *Severity* rating of each finding is determined as a synthesis of two sub-ratings: *Impact* and *Likelihood*. It ranges from *Informational* to *Critical*.

If we have found a scenario in which an issue is exploitable, it will be assigned an impact rating of *High*, *Medium*, or *Low*, based on the direness of the consequences it has on the system. If we haven't found a way, or the issue is only exploitable given a change in *configuration* (system settings or parameters, such as deployment scripts, compiler configurations, using multi-signature wallets for owners, etc.) or given a change in the codebase, then it will be assigned an impact rating of *Warning* or *Info*.

Low to *High* impact issues also have a *Likelihood*, which measures the probability of exploitability during runtime.

The full definitions are as follows:

Severity

		Likelihood			
		High	**Medium**	**Low**	**N/A**
Impact	**High**	Critical	High	Medium	-
	Medium	High	Medium	Low	-
	Low	Medium	Low	Low	-
	Warning	-	-	-	Warning
	Info	-	-	-	Info

Table 1. Severity of findings

Impact

- **High** - Code that activates the issue will lead to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue will result in consequences of serious substance.
- **Low** - Code that activates the issue will have outcomes on the system that are either recoverable or don't jeopardize its regular functioning.
- **Warning** - The issue cannot be exploited given the current code and/or `configuration`, but could be a security vulnerability if these were to change slightly. If we haven't found a way to exploit the issue given the time constraints, it might be marked as a "Warning" or higher, based on our best estimate of whether it is currently exploitable.
- **Info** - The issue is on the borderline between code quality and security. Examples include insufficient logging for critical operations. Another example is that the issue would be security-related if code or `configuration` was to change.

Likelihood

- **High** - The issue is exploitable by virtually anyone under virtually any circumstance.
- **Medium** - Exploiting the issue currently requires non-trivial preconditions.
- **Low** - Exploiting the issue requires strict preconditions.

2.4. Review Team

The following table lists all contributors to this report. For authors of the specific revision, see the “Revision team” section in the respective “Report revision” chapter.

Member’s Name	Position
Andrej Lukačovič	Lead Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

2.5. Disclaimer

We’ve put our best effort to find all vulnerabilities in the system, however our findings shouldn’t be considered as a complete list of all existing issues. The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

3. Executive Summary

Jpegs.fun is a protocol that allows bonding curve trading of Non-Fungible-Tokens (NFTs) before the liquidity is migrated to the Tensor Automated Market Maker (AMM).

Revision 1.0

Jpegs.fun engaged Ackee Blockchain Security to perform a security review of the Jpegs.fun protocol with a total time donation of 4 engineering days in a period between November 25 and December 2, 2024, with Andrej Lukačovič as the lead auditor.

The audit was performed on commits `20f612`[1] and `52007f`[2], with the following scope:

- Bonding Curve Market
- Migrator

We began our review by analyzing the provided documentation and source code. After examining the tests and verifying the program accounts, we conducted a deep dive into the protocol's functionality. This included writing proof-of-concept tests, independent protocol testing, vulnerability assessment, Cross Program Invocation analysis, and basic fuzzing of the bonding curve mathematics.

During the review, we paid special attention to:

- ensuring the accounts configuring the bonding curve are managed exclusively by authorized parties;
- verifying the main configuration cannot be re-initialized;
- validating all bonding curve and trading parameters are correctly set;

- confirming the buy and sell methods compute purchase costs accurately;
- preventing unauthorized user purchases;
- verifying fees are correctly applied to purchase collateral;
- ensuring all variables update properly during trading;
- confirming secure migration handling for transferring funds out of the bonding curve.

Our review resulted in 10 findings, ranging from Warning to Critical severity. The most severe one is C1.

Ackee Blockchain Security recommends Jpegs.fun:

- address all reported issues;
- implement sufficient validation for all configuration parameters;
- ensure authority over the config can be transferred securely;
- ensure assets are directly migrated to the accounts used within the Migrator program;
- ensure both programs are thoroughly tested, including edge cases and advanced scenarios;
- ensure there are no significant price differences between the Tensor AMM and the bonding curve;
- improve the overall code quality, including modularization, consistency, and documentation;
- avoid deploying directly to mainnet and proceeding to production phase until code quality is improved and testing is more robust, as the current state of the source code indicates potential challenges in responding to production issues securely.

See Report Revision 1.0 for the system overview and trust model.

Revision 2.0

Jpegs.fun engaged Ackee Blockchain Security to perform a security review of the Jpegs.fun protocol with a total time donation of 2.5 engineering days in a period between December 17 and December 20, 2024, with Andrej Lukačovič as the lead auditor.

The audit was performed on the commits `72d4ac`[3] and `d4122b`[4], and the scope was the following:

- Bonding Curve Market
- Migrator

During the second revision additional issues were discovered and not all reported issues were resolved.

Our review resulted in 7 findings, ranging from Warning to Critical severity. The most severe one is C2.

Ackee Blockchain Security recommends Jpegs.fun:

- address all unresolved issues and perform another audit revision. The protocol is not ready for deployment in the current state.

See Report Revision 2.0 for the system overview and trust model.

Revision 2.1

The fix review was performed on the commits `9ae667`[5] and `f7de11`[6] respectively. The scope remains the same, namely:

- Bonding Curve Market
- Migrator

Findings C2, W5, W6, and W7 were fixed. Findings M4, M5 and L1, were acknowledged by the client. Additionally, findings M3 and W1 remain partially fixed.

Ackee Blockchain Security recommends Jpegs.fun:

- address all unresolved issues, including those reported in 1.0 and 2.0;
- improve the overall code quality of both programs;
- ensure that both programs contain robust tests that can be executed locally;
- avoid deploying Solana programs to the mainnet and proceeding with the production phase until all issues are fixed and both Solana programs are well tested;
- perform another audit revision after resolving all issues and improving code quality and tests.

See Report Revision 2.1 for the system overview and trust model.

[1] full commit hash: `20f6127c1666064e69dc8deac2a966a7d38ea861`

[2] full commit hash: `52007f0068ed249c7162176bb018ad3d0d9a70d4`

[3] full commit hash: `72d4aca00134eb2516f0f96c8be8eb6f18055e87`

[4] full commit hash: `d4122bd2d1e58079cfc4b9434600e01866692833`

[5] full commit hash: `9ae667323d4889bf779f87d20ff8e68345313593`

[6] full commit hash: `f7de116a3d4cd056b2e7ad2073215f694b745327`

4. Findings Summary

The following section summarizes findings we identified during our review. Unless overridden for purposes of readability, each finding contains:

- *Description*
- *Exploit scenario* (if severity is low or higher)
- *Recommendation*
- *Fix* (if applicable).

Summary of findings:

Critical	High	Medium	Low	Warning	Info	Total
2	2	5	1	7	0	17

Table 2. Findings Count by Severity

Findings in detail:

Finding title	Severity	Reported	Status
C1: Collateral migration is not possible if the reserves exceed 3 SOL	Critical	1.0	Fixed
H1: Possible inadequate trading fee	High	1.0	Fixed
H2: Migration account inconsistencies in liquidity transfers	High	1.0	Partially fixed
M1: Inability to update config admin	Medium	1.0	Fixed

Finding title	Severity	Reported	Status
M2: Price difference between bonding curve and Tensor AMM	Medium	1.0	Partially fixed
M3: Possible inability to migrate funds as migrator is not marked as mutable	Medium	1.0	Partially fixed
W1: Poor code quality	Warning	1.0	Partially fixed
W2: Lack of quality testing	Warning	1.0	Acknowledged
W3: Possibility for fee to be zero	Warning	1.0	Acknowledged
W4: Potential frontrunning of initialization	Warning	1.0	Acknowledged
C2: The authority transfer process cannot be finalized	Critical	2.0	Fixed
M4: Migration account inconsistencies	Medium	2.0	Acknowledged
M5: Protocol owner has the ability to deposit an arbitrary amount of liquidity	Medium	2.0	Acknowledged
L1: Price difference between bonding curve and Tensor AMM	Low	2.0	Acknowledged
W5: Possible inability to create Tensor AMM pool	Warning	2.0	Fixed
W6: Closing ticket account is not implemented correctly	Warning	2.0	Fixed

Finding title	Severity	Reported	Status
W7: Risk of incorrectly setting market config parameters	Warning	2.0	Fixed

Table 3. Table of Findings

Report Revision 1.0

Revision Team

Member's Name	Position
Andrej Lukačovič	Lead Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

Jpegs.fun Launchpad is a project consisting of two Solana programs, Market and Migrator. The Market program is responsible for managing the logic of the bonding curve before Non-Fungible Tokens (NFTs) and collateral are migrated to the Tensor Automated Market Maker (AMM). The Migrator program handles the logic dedicated to the migration of assets.

Trust Model

The Market program is managed by an admin, who has access to configure fields such as bonding curve parameters, fees, and migration accounts. However, the audit revealed poor handling of these configurations. For example, there is no validation for parameters, allowing the admin to set a fee to any value, even exceeding 100%.

Once the bonding curve phase is complete, assets (collateral and NFTs) are transferred "out" of the bonding curve to allow the Migrator program to move them to the Tensor AMM pool. During the migration phase, the accounts holding the assets have control over them. It is the client's responsibility to ensure the secure migration of these assets to the Tensor AMM.

Findings

The following section presents the list of findings discovered in this revision.

C1: Collateral migration is not possible if the reserves exceed 3 SOL

Critical severity issue

Impact:	High	Likelihood:	High
Target:	market/lib.rs	Type:	Denial of service

Description

The migration of assets is divided into two parts: the migration of collateral and the migration of remaining Non-Fungible Tokens (NFTs). The collateral migration process begins by checking that the current bonding curve balance is below 3 SOL.

The collection is configured for 6,000 Non-Fungible Tokens (NFTs), with 5,400 tokens available for trading and 600 tokens allocated as liquidity for the Tensor Automated Market Maker (AMM). The bonding curve trading uses the following parameters:

- bondingK: 1,500,000,000,000,000;
- bondingA: 1,000,000;
- bondingX0: 9,500.

The NFT pricing structure is as follows:

- Total cost for 5,400 NFTs: 62,623,595,903 lamports (~62 SOL).

The following code snippet demonstrates an incorrect implementation. Specifically, the second part of the condition `bonding_curve.to_account_info().lamports() < 3_000_000_000`.

Listing 1. Excerpt from Market

```
require!(bonding_curve.current_supply == bonding_curve.total_sale_token_amount
    && bonding_curve.to_account_info().lamports() < 3_000_000_000,
    ErrorCode::NotTimeToWithdraw
);
```

Exploit scenario

Migration is not possible as the final collateral amount (~ 62 SOL) is higher than 3 SOL.

Recommendation

Update the direction of the comparison sign in the `require!` implementation or completely remove the condition. The amount of final collateral liquidity can and should be validated when setting the bonding curve parameters.

Fix 2.0

The issue has been resolved by updating the comparison sign.

Go back to Findings Summary

H1: Possible inadequate trading fee

High severity issue

Impact:	High	Likelihood:	Medium
Target:	market/lib.rs	Type:	Configuration

Description

The program lacks parameter validation for trading fees set by the admin. Without proper bounds checking on the `fee_bps` parameter, fees could be set to exceed 100%. This could result in users paying more than double the intended trading amount.

The following code snippet demonstrates the absence of validation checks for the `fee_bps` variable in the Config:

Listing 2. Excerpt from Market

```
    pub fn initialize(
        ctx: Context<Initialize>,
        /// ...
        fee_bps: u64,
        /// ...
    ) -> Result<()> {
        let market = &mut ctx.accounts.market;

        /// ...

        market.fee_bps = fee_bps;

        /// ...
        Ok(())
    }
```

Exploit scenario

Alice, an admin, sets `fee_bps` to a value exceeding 100%. Bob, a user, attempts to trade and pays a fee greater than his collateral amount.

Recommendation

Implement parameter validation to restrict `fee_bps` within an acceptable range. Add a maximum fee limit of 10% to protect users from excessive fees.

Fix 2.0

The issue has been fixed, and the fee is now limited to 10%.

Go back to Findings Summary

H2: Migration account inconsistencies in liquidity transfers

High severity issue

Impact:	High	Likelihood:	Medium
Target:	market/lib.rs, migrator/lib.rs	Type:	Data validation

Description

The Migration program manages Tensor Automated Market Maker (AMM) pool creation and liquidity depositing. The pool owner, a Program Derived Address (PDA) derived from "owner" and the Migrator program ID, is responsible for both collateral and Non-Fungible Token (NFT) liquidity deposits.

The Market program implements two inconsistent migration paths:

- collateral liquidity migrates to the migrator account (transaction signer). Since Program Derived Addresses (PDAs) support transaction signing only through Cross Program Invocations (CPIs), which are not implemented, assets must follow a two-step migration: first to the migrator account, then to the pool owner for liquidity pool deposit;
- NFT liquidity migrates to the `migrate_pool_owner` account. While this address is stored in the Configuration, the program performs no address validation, creating potential risks of theft or accidental asset migration to incorrect accounts.

Exploit scenario

Alice, the protocol owner, can steal assets or accidentally transfer them to an unintended account due to the lack of address validation in the migration process. Bob, the asset holder, risks losing their assets during the migration.

Recommendation

Implement the following changes:

- migrate both collateral and NFT liquidity to the `migrate_pool_owner` account;
- validate the `migrate_pool_owner` address using the seeds;
- implement Cross Program Invocation (CPI) between the Market and Migrator for atomic migration execution.

Partial solution 2.0

See M4.

Go back to Findings Summary

M1: Inability to update config admin

Medium severity issue

Impact:	Medium	Likelihood:	Medium
Target:	market/lib.rs, migrator/lib.rs	Type:	Configuration

Description

The configuration parameters for trading can be updated; however, the `admin` field in the configuration cannot be modified. This limitation poses a significant security risk, as the ability to update the admin field is crucial for mitigating unauthorized parameter manipulation in cases of private key compromise.

Exploit scenario

Alice is the current admin of the protocol. Her private key becomes compromised, and Bob gains unauthorized access. Due to the inability to update the admin field, Alice cannot revoke Bob's access to critical protocol functions. Bob can now:

- modify fee recipient accounts;
- alter migration parameters;
- potentially cause loss of protocol funds.

Recommendation

Implement functionality to update the admin field in the Config Account. The implementation should follow a two-step authority transfer process:

- the current authority initiates the transfer;
- the target authority must claim the access;
- the initial authority retains revocation rights before claim finalization.

This approach mitigates the risk of accidental authority transfer to incorrect addresses.

Partial solution 2.0

See C2.

Fix 2.1

The issue was fixed by adding a two-step authority transfer process.

Go back to Findings Summary

M2: Price difference between bonding curve and Tensor AMM

Medium severity issue

Impact:	Low	Likelihood:	High
Target:	market/lib.rs, migrator/lib.rs	Type:	Configuration

Description

The Non-Fungible Token collection created in the Market program is configured for 6,000 Non-Fungible Tokens (NFTs), with 5,400 available for trading and 600 allocated as liquidity for the Tensor Automated Market Maker (AMM). The bonding curve trading uses the following parameters:

- bondingK: 1,500,000,000,000,000
- bondingA: 1,000,000
- bondingX0: 9,500.

The NFT pricing structure is as follows:

- First NFT purchase price: 7,756,906 lamports
- Last NFT purchase price: 17,618,750 lamports (0.01761875 SOL)
- Total cost for 5,400 NFTs: 62,623,595,903 lamports (~62 SOL).

The Tensor AMM pool's initial price is set to 0.01 SOL, resulting in a 43% loss for the final Market NFT buyer after liquidity migration.

Exploit scenario

Alice is the last buyer of an NFT through the bonding curve mechanism, paying 0.01761875 SOL. When the protocol migrates liquidity to the Tensor AMM, the price is set to 0.01 SOL. As a result, Alice immediately suffers a 43% loss on her investment.

Recommendation

Configure the initial price in the Tensor AMM to match or slightly exceed the final price in the bonding curve phase.

Partial solution 2.0

See L1.

Go back to Findings Summary

M3: Possible inability to migrate funds as migrator is not marked as mutable

Medium severity issue

Impact:	Medium	Likelihood:	Medium
Target:	market/lib.rs	Type:	Denial of service

Description

The `migrate_withdraw` and `migrate_mint` functions require a `migrator` account that must be mutable to perform their operations:

- In `migrate_withdraw`, the `migrator` account receives transferred collateral liquidity;
- In `migrate_mint`, the `migrator` account pays fees for Metaplex Core Asset creation.

However, the `migrator` account in both `MigrationMint` and `Withdraw` contexts lacks the mutable flag. This creates the following issues:

1. the migration succeeds only when the `migrator` account is the Transaction Fee payer (as fee payers are mutable by default);
2. the migration fails when using a different Transaction Fee payer account, as the `migrator` account remains immutable.

While manually marking the account as writable in the IDL provides a temporary solution, this approach is not recommended because:

- it requires manual updates during each IDL regeneration;
- the Cross Program Invocation (CPI) code generated by Anchor would not mark this account as mutable, limiting CPI functionality.

Exploit scenario

In case a different transaction fee payer is used instead of the `migrator`, the client will not be able to migrate until the migrator account in the input is marked as writable.

Recommendation

Mark the `migrator` account as mutable in both `MigrationMint` and `Withdraw` contexts.

Partial solution 2.0

The `migrator` account in the `MigrationMint` context remains immutable while serving as the fee payer for NFT creation. Therefore, the issue is Partially Fixed.

Partial solution 2.1

The `migrator` account in the `MigrationMint` context remains immutable while serving as the fee payer for NFT creation. Therefore, the issue is Partially Fixed.

Go back to Findings Summary

W1: Poor code quality

Impact:	Warning	Likelihood:	N/A
Target:	market/lib.rs, migrator/lib.rs	Type:	Code quality

Description

The whole source code for both Solana programs demonstrates poor code quality. For example, the following areas can be improved:

- everything is written in one file;
- the source code is inconsistent;
- the source code should be better commented;
- the source code lacks checks, for example, there are plenty of raw AccountInfos. Even though the accounts are mostly used for Cross Program Invocations (CPI), relying completely on the fact that the target program will check the accounts is neither secure nor recommended. The same applies to Configuration data validations;
- updating fields in Configuration requires re-entering all other fields. Instead, specify the Instruction inputs as optional;
- multiple contexts contain unnecessary accounts in their inputs. These can, for example, be passed simply as Public Keys in Instruction inputs;
- there are multiple parts of code and instruction inputs that might be unnecessary;
- allocate more space in Configuration and Bonding Curve for future features;
- unknown subtraction numbers like -100 are used during collateral migration;
- ticket accounts can be closed if not needed anymore, saving funds of users;

- missing program ID validations; even though program IDs are currently validated in the SDK, it is recommended to also validate program IDs within your programs;
- there are many potential unhandled panics.

Recommendation

Implement modifications to the source code to address the issues outlined above.

Partial solution 2.0

The code quality has improved, but there is still room for further enhancements.

- many parts of the source code remain unused;
- the modularization is inadequate;
- source code is commented out;
- checked arithmetic is not used;
- `pub pending_admin: Option<Pubkey>, // 32 bytes` the comment is misleading, the `Option<Pubkey>` is 33 bytes long not 32. See the Reference

Partial solution 2.1

The code quality has improved, but there is still room for further enhancements:

- many parts of the source code remain unused;
- the modularization is inadequate;
- source code is commented out;
- checked arithmetic is not utilized;

- `pub pending_admin: Option<Pubkey>, // 32 bytes` the comment is misleading; the `Option<Pubkey>` is 33 bytes long, not 32; See the Reference;
- some accounts are unnecessarily marked as mutable.

Go back to Findings Summary

W2: Lack of quality testing

Impact:	Warning	Likelihood:	N/A
Target:	market/lib.rs, migrator/lib.rs	Type:	Code quality

Description

Both Solana programs are poorly tested. The tests include only happy path scenarios, verifying basic functionality of the programs. There is no evidence that the two programs were tested together either locally or on devnet. Additionally, advanced testing, such as fuzzing the bonding curve math function, is lacking. This is particularly concerning as the function may panic when different trading parameters are used.

Recommendation

Implement additional tests, test unhappy paths and ensure both programs are tested together locally and on devnet. Use sophisticated testing techniques, such as fuzzing, to extensively validate the functionality of the bonding curve calculations.

Acknowledgment 2.0

The client provided the following statement:

> *For W2, we have conducted extensive quality testing on devnet through hundreds of program versions deployed by the '5EDuQNiCKKQX2miovFD8KZsH83qaiLgoy4wBLzX8Bsah' account and thousands of transactions. Specifically, numerical parameters like bonding curve values are thoroughly validated outside the contract, tested on the testnet, and then deployed to the mainnet with the same values to mitigate the risk of incorrect bonding curve parameters being applied.*
>
> — Jpegs.fun Team

While testing on the devnet is highly recommended, the local tests are of poor quality, with significant number of them commented out. Testing locally is important to build a strong tests foundation for future releases and to easily verify new bonding curve trading parameters if updates are required.

Acknowledgment 2.1

The quality of the tests has not improved. The TypeScript tests are still mostly commented out. Although the client's response (see Acknowledgement 2.0) contains a description of testing on the devnet, this is insufficient; tests that can be run locally must be present. Furthermore, these local tests need to be robust and test both unhappy paths and happy scenarios.

Additionally, tests that ensure the correctness of migration are absent. In other words, as the scope includes two Solana Programs, the tests must ensure that both programs are tested together, again in both happy and unhappy scenarios.

Go back to Findings Summary

W3: Possibility for fee to be zero

Impact:	Warning	Likelihood:	N/A
Target:	market/lib.rs	Type:	Data validation

Description

With the following trading parameters:

- bondingK: 1,500,000,000,000,000;
- bondingA: 1,000,000;
- bondingX0: 9,500.

The fee will not be zero since the starting price is sufficiently high (7,756,906 lamports).

However, if the bonding curve parameters change, the fee computation could result in a zero fee for some users.

The following code demonstrates how the fee is calculated. If `total_price * market.fee_bps` is smaller than 10,000, the integer arithmetic will result in zero, leading to a final fee of zero.

Listing 3. Excerpt from Market

```
let fee = total_price * market.fee_bps / 10_000;
```

Recommendation

Ensure there is a minimum fee in case the formula results in zero.

Acknowledgment 2.0

The client provided the following statement:

> *Regarding W3, due to business requirements, the minimum price of the token is always greater than 0.001 SOL. This ensures that total_price * market.fee_bps will always be greater than 10,000.*
>
> — Jpegs.fun Team

However, the point highlighted by the client is not enforced in the Market program.

Acknowledgment 2.1

The client did not provide any additional comments, and the issue remains unresolved.

Go back to Findings Summary

W4: Potential frontrunning of initialization

Impact:	Warning	Likelihood:	N/A
Target:	market/lib.rs, migrator/lib.rs	Type:	Data validation

Description

Both programs contain initialization processes that are vulnerable to frontrunning, which can lead to undesired situations.

If an attacker discovers that the program has been deployed but not yet initialized, they can call the initialization function, which will be processed successfully the first time. This would prevent the client (i.e., the desired admin) from initializing the configuration, leading to unintended and potentially undesired scenarios.

Recommendation

This attack is rather exotic, so it is classified as a Warning. However, it can be easily prevented, for example, by implementing the following code snippet:

Listing 4. Example code snippet

```
#[account(
    constraint = admin.key()
    == program_data.upgrade_authority_address.unwrap_or_default())]
admin: Signer<'info>,

#[account(
    seeds = [crate::ID.as_ref()],
    bump,
    seeds::program = bpf_loader_upgradeable::id(),
)]
program_data: Account<'info, ProgramData>,

#[account(
    init,
    space = ...,
    payer = admin,
```

```
        seeds = [...],
        bump
    )]
    config: Account<'info, Config>,
```

The code above demonstrates how to ensure that only the Update Authority of the Solana program can perform the initialization.

Acknowledgment 2.0

The client provided the following statement:

> *For W4, since the program will be pre-configured before the service launch, we are confident that no issues will arise.*
>
> — Jpegs.fun Team

Even though the likelihood of such an attack might be low, we cannot conclude that the issue is fixed based on the client's response. For example, the issue could be avoided if the deployment of the Solana program and the initialization of the market account are performed together in one transaction, thereby preventing frontrunning. While this might be ensured on the off-chain side, as stated above, we cannot make such assumption.

Acknowledgment 2.1

The client did not provide any additional comments, and the issue remains unresolved.

Go back to Findings Summary

Report Revision 2.0

Revision Team

Member's Name	Position
Andrej Lukačovič	Lead Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

In alignment with the previous version.

Trust Model

The trust model remains the same. Considerations from the previous revision have improved; data validation in the Market program has been enhanced, and the temporary account responsible for handling liquidity between programs is now better defined. However, some considerations are still recommended to be addressed to ensure the complete correctness and security of the programs.

Findings

The following section presents the list of findings discovered in this revision.

C2: The authority transfer process cannot be finalized

Critical severity issue

Impact:	High	Likelihood:	High
Target:	market/lib.rs, migrator/lib.rs	Type:	Logic error

Description

Although the two-step authority transfer process mentioned in M1 was implemented, the implementation is incorrect and cannot be finalized.

Exploit scenario

Alice, the current admin, initiates the authority transfer process. However, the constraint `constraint = new_admin.key() != market.pending_admin.unwrap()` prevents Bob, the new admin, from accepting the transfer. As a result, the transfer cannot be finalized, leaving the authority in the hands of Alice.

Recommendation

Change the constraint to `constraint = new_admin.key() == market.pending_admin.unwrap()`.

Fix 2.1

The issue was fixed in line with the recommendation.

An additional recommendation that would improve the quality of the code is to add a check that verifies if the account that finalizes the admin transfer is the current admin. If this condition is met, the process will reset, turning the pending authority to None. This is an improvement because currently, if the pending admin is set incorrectly, it cannot be reverted to None; it can only be

accepted or updated to a different address.

Go back to Findings Summary

M4: Migration account inconsistencies

Medium severity issue

Impact:	High	Likelihood:	Low
Target:	market/lib.rs, migrator/lib.rs	Type:	Data validation

Description

The issue H2 from the previous version was partially fixed. While the program now stores the address of the `MigratorPoolOwner` responsible for depositing liquidity to the Tensor Automated Market Maker pools, the implementation is still not completely correct. The following code snippet shows the new logic for how the reference address is stored:

Listing 5. Excerpt from Market

```
if let (Some(migrate_pool_owner), Some(program))
    = (migrate_pool_owner, migration_program) {

    let (calculated_migrate_pool_owner, _bump)
        = Pubkey::find_program_address(&[b"owner"], &program);

    require!(
        calculated_migrate_pool_owner == migrate_pool_owner,
        ErrorCode::InvalidMigratePoolOwnerAddress
    );

    market.migrate_pool_owner = migrate_pool_owner;
}
```

The problem is that the Migrator Program ID is fully customizable in the Initialize Instruction input, which is unnecessary and can additionally cause security issues.

Exploit scenario

- Alice is an attacker;
- Alice can be the protocol owner or an attacker who possesses the Market account admin authority private key (for example, due to private key leakage);
- Alice creates an additional program called Migrator-Malicious that will transfer funds not to the Tensor Automated Market Maker pool but to the wallet she controls.

Since the Migrator Program ID is customizable in the Initialize instruction, Alice can update the `MigratorPoolOwner` address using not the Migrator Program ID, but the Migrator-Malicious program ID.

Next, as the `Withdraw` and `MigrationMint` instructions check that the `MigratorPoolOwner` is the same as the `MigratorPoolOwner` stored in the Market account, the `Withdraw` and `MigrationMint` will transfer assets to the updated `MigratorPoolOwner` account.

Lastly, the updated `MigratorPoolOwner` account will be used in the Migrator-Malicious program to transfer assets to Alice's wallet.

Recommendation

Ensure the Migrator Program ID is not customizable. If the Migrator Program ID is not customizable but constant, it is unnecessary to store the `MigratorPoolOwner` address in the Market account at all.

Use logic where the address of the `MigratorPoolOwner` is derived directly in the corresponding instruction context with the specified seeds and the corresponding Migrator Program ID, which in this case will be constant.

Acknowledgment 2.1

The client provided the following statement:

> *The feedback suggests not storing the PoolOwner in the Market and not making the Migrator program ID customizable. Instead, it recommends using seeds for derivation. However, there are constraints that make this approach infeasible. The PoolOwner account is a PDA account that must be used as a signer for migration within the Migrator program. Since it is derived using the Migrator program ID, the Market must either store the Migrator program ID to derive the PoolOwner or, alternatively, store the PoolOwner itself instead of the Migrator program ID as configuration data. The reason for receiving the Migrator program ID as input in the Initialize() function is to enable additional verification of the PoolOwner.*
>
> — Jpegs.fun Team

The issue is still present, and no fixes were provided.

Go back to Findings Summary

M5: Protocol owner has the ability to deposit an arbitrary amount of liquidity

Medium severity issue

Impact:	High	Likelihood:	Low
Target:	migrator/lib.rs	Type:	Logic error

Description

The Migrator program expects the `deposit_sol` instruction input to specify the amount of liquidity (SOL) to be added to the Tensor Automated Market Maker pool. However, there is no restriction on the protocol owner selecting an arbitrary amount of tokens, potentially leading to a situation where not all liquidity is deposited.

The following code snippet contains logic where the amount is taken from the instruction input:

Listing 6. Excerpt from Migrator

```
pub fn deposit_sol(ctx: Context<DepositSol>, amount: u64) -> Result<()> {
// ...
let deposit_sol_ix = DepositSolBuilder::new()
    .owner(ctx.accounts.owner.key())
    .pool(ctx.accounts.pool.key())
    .system_program(ctx.accounts.system_program.key())
    .lamports(amount) /// arbitrary amount from the instruction input
    .instruction();

}
// ...
```

Exploit scenario

- Alice is the authority who has access to create pools and deposit liquidity using the Migrator program;
- Alice can be the protocol owner or an attacker who possesses the Migrator account admin authority private key (for example, due to private key leakage);
- Alice has the ability to update the Migrator program.

Alice can deposit an arbitrary amount of liquidity into the Tensor Automated Market Maker pool. Alice kept funds, intentionally or mistakenly, in the `MigratorPoolOwner` account, which were meant to be transferred to the Tensor Automated Market Maker pool. Alice updates the Migrator program with logic that allows transferring kept funds from the `MigratorPoolOwner` to a wallet controlled by Alice, effectively resulting in the theft of part of the liquidity that was collected during the bonding curve phase.

Recommendation

Deposit all available liquidity that has been migrated to the temporary account by using the account's balance, rather than an arbitrary number from the instruction input.

Acknowledgment 2.1

The client provided the following statement:

> *If we assume a scenario where only a single pool is being migrated, it might be better to follow the report's suggestion of transferring the entire balance held in the PoolOwner. However, the PoolOwner is also used when migrating multiple NFT collections simultaneously. In such cases, the PoolOwner balance may include funds from different pools.*

> *Also, since minting costs can vary by NFT collection, it is challenging to fix or calculate the deposit amount programmatically. The control of migration and state management should be handled by the central server using the Migrator program, and the deposit amount must be set flexibly by the central server according to each pool with considering upper two reasons.*
>
> — Jpegs.fun Team

We agree with the client's statement that transferring the whole balance in the `MigratorPoolOwner` account is not a good solution within the current implementation.

However, the issue remains present, and maintaining a flexible deposited amount is not a viable solution. The protocol must operate in a trustless manner, and there is no room for flexibility and central control when transferring funds from the `MigratorPoolOwner`.

One of the solutions is to make the `MigratorPoolOwner` unique for each new collection created during the `CreateCollection` instruction. The collection mint address will be part of the seeds for the `MigratorPoolOwner`. Then, as each collection has its corresponding `MigratorPoolOwner`, all of the liquidity can be transferred.

Go back to Findings Summary

L1: Price difference between bonding curve and Tensor AMM

Low severity issue

Impact:	Low	Likelihood:	Medium
Target:	market/lib.rs, migrator/lib.rs	Type:	Configuration

Description

Even though the client updated the source code based on the reported issue M2, complete security is still not ensured.

For reference, the following code snippet provides the logic for setting the initial price:

Listing 7. Excerpt from Migrator

```
pub fn create_pool(
    ctx: Context<CreatePool>,
    starting_price: u64, ...
) -> Result<()> {
    //...
    require!(
        starting_price > 0 && starting_price < 10_000_000_000,
        ErrorCode::InvalidStartingPrice
    );
    let pool_config = PoolConfig {
        // ...
        starting_price,
        // ...
    };
    //...
}
```

Although the check that the `starting_price` cannot be higher than 10 SOL was introduced, this does not fix the issue reported as M2.

Exploit scenario

- Alice is an attacker.
- Alice is the authority who has access to create pools using the Migrator program.
- Alice can be the protocol owner or an attacker who possesses the Migrator account admin authority private key (for example, due to private key leakage).

Scenario 1

- Bob is a user who participated in the later stage of the bonding curve.

Alice sets the `starting_price` intentionally or mistakenly lower than the price in the later stage of the bonding curve. Bob's assets lose value due to the price difference.

Scenario 2

- Alice participates in the early stage of the bonding curve.

Alice sets the `starting_price` intentionally higher than the price in the later stage of the bonding curve. This will be possible if the bonding curve price is not higher than 10 SOL. Alice's assets gain value due to the price difference.

Recommendation

Ensure that the price remains the same or slightly higher (less than 1%) after the migration. This can be achieved, for example, by deriving the price within the Migrator program.

Acknowledgment 2.1

The client provided the following statement:

> *As mentioned in the report, if the pool's starting price in Tensor AMM is lower than the Bonding Curve price in the Market program, some users may incur losses. Our team is fully aware of this. Typically, the starting price of pools created in Tensor AMM is set higher than the last price on the Bonding Curve in the Market program. However, while keeping it in mind, the pricing of pools in Tensor AMM needs to remain flexible to meet our business requirement.*
>
> — Jpegs.fun Team

The issue was acknowledged by the client. The issue remains present, and users must be informed about the potential loss of asset value.

Go back to Findings Summary

W5: Possible inability to create Tensor AMM pool

Impact:	Warning	Likelihood:	N/A
Target:	migrator/lib.rs	Type:	Logic error

Description

The Migrator program's initialize instruction contains two input arguments: `migrate_bump: u8` and `owner_bump: u8`. Not only is `migrate_bump` unused, but it is also strongly discouraged to obtain bumps from instruction inputs.

If the `owner_bump` is set incorrectly, it will not be possible to create new pool using the Tensor Automated Market Maker due to the following line:

Listing 8. Excerpt from Migrator

```
let seeds = &[b"owner".as_ref(), &[migrate.owner_bump]];
```

Implemented logic will derive incorrect seeds resulting in `Cross-program invocation with unauthorized signer or writable account.` error.

Recommendation

Obtain the bumps directly from the account context, where they are handled by the Anchor Framework, example in the following code snippet:

Listing 9. Excerpt from Migrator

```
let seeds = &[b"owner".as_ref(), &[ctx.bumps.owner]];
```

Fix 2.1

The issue was fixed in line with the recommendation.

Go back to Findings Summary

W6: Closing ticket account is not implemented correctly

Impact:	Warning	Likelihood:	N/A
Target:	market/lib.rs	Type:	Logic error

Description

Subtracting lamports is not completely correct logic for closing account. Closing account consist of subtracting lamports, clearing data and changing the account owner.

Recommendation

Instead of manually subtracting lamports, use the `close` constraint for the ticket account.

Fix 2.1

The issue was fixed in line with the recommendation.

Go back to Findings Summary

W7: Risk of incorrectly setting market config parameters

Impact:	Warning	Likelihood:	N/A
Target:	market/lib.rs	Type:	Data validation

Description

The market config initialization instruction in the Market program allows for instruction inputs to be optional. While this is expected, as the admin should be able to update the config fields without entering all other fields in order to update only one field, this approach is not correctly implemented.

During the first invocation of the config initialization, all fields should be required. This means none of the instruction inputs should be optional. The issue that can arise is that if the input is missing, the initialization will still pass. If the corresponding field has not been set yet, the field value defaults to 0 (as these are the default values during account data allocation). This could result in situations where, for example, the bonding curve trading parameter is not set, thus defaulting to 0.

Recommendation

During the first initialization of the market config, all fields must be required. If the market config is already initialized, the fields can be optional.

Fix 2.1

The issue was fixed in line with the recommendation.

Go back to Findings Summary

Report Revision 2.1

Revision Team

Member's Name	Position
Andrej Lukačovič	Lead Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

Apart from the fixes of issues reported in the previous version, there are no additional changes to the Solana programs. Thus, the system overview is consistent with the previous version.

Trust Model

Users must trust the protocol owner to ensure that all assets are migrated correctly to the Tensor Automated Market Maker, as the protocol owner determines the amount to be transferred. Users must also trust the protocol owner to set the price at the Tensor Automated Market Maker correctly, as the protocol owner decides the initial price. Users must trust the protocol owner that the assets are transferred to the correct `MigratorPoolOwner` account, which is later responsible for transferring assets to the Tensor Automated Market Maker.

Appendix A: How to cite

Please cite this document as:

Ackee Blockchain Security, Jpegs.fun: Launchpad, 7.1.2025.

ackee
blockchain security

Thank You

Ackee Blockchain a.s.

Rohanske nabrezi 717/4
186 00 Prague
Czech Republic

hello@ackee.xyz